НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Методы обработки изображений

Лабораторная работа № 1

Вычисление яркости в точках на поверхности.

Выполнил студент:

Белик Тимур

Группа № P3314

Преподаватель:

Потемин Игорь Станиславович

г. Санкт-Петербург 2026

Цель

Изучить, как вычисляется яркость в точках на плоскости треугольника с учетом освещения от точечных источников света и диффузно отражательных свойств поверхности.

Используемые формулы

1. «Цветная» сила излучения источника под углом к оси источника света:

$I\left(RGB, \vec{s}\right) = I_0(RGB) \cos cos\, \theta$, *где* $I_0(RGB)$ *– «цветная» сила излучения источника света в направлении его оси* $\vec{O}, \|\vec{O}\| = 1$, θ *- угол между направлением распространения света и осью источника света,* $\cos cos\, \theta$ *– диаграмма излучения.*

2. «Цветная» освещенность точки:

$E\left(RGB, \vec{P}_T\right) = \frac{I\left(RGB, \vec{s}\right) cos\alpha}{R^2}$, *где* $I\left(RGB, \vec{s}\right)$ *– «цветная» сила излучения света,* α *- угол между направлением света и нормалью к освещаемой поверхности,* R *- расстояние от источника света до рассматриваемой точки.*

3. Перевод локальных координат точки в плоскости в глобальные

$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y\right)$, *где* x *и* y *смещения по ребрам треугольника.*

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{\left(\vec{P}_2 - \vec{P}_0\right) \times \left(\vec{P}_1 - \vec{P}_0\right)}{\|\left(\vec{P}_1 - \vec{P}_0\right) \times \left(\vec{P}_2 - \vec{P}_0\right)\|}$$

5. Вектор от точки плоскости до источника света:

$\vec{s} = \vec{P}_T - \vec{P}_L$, $R^2 = \|\vec{s}\|^2$, $\cos cos\,\alpha = \frac{\vec{s} \bullet \vec{N}}{\|\vec{s}\|}$, $\cos cos\,\theta = \frac{\vec{s} \bullet \vec{O}}{\|\vec{s}\|}$

6. Яркость точки:

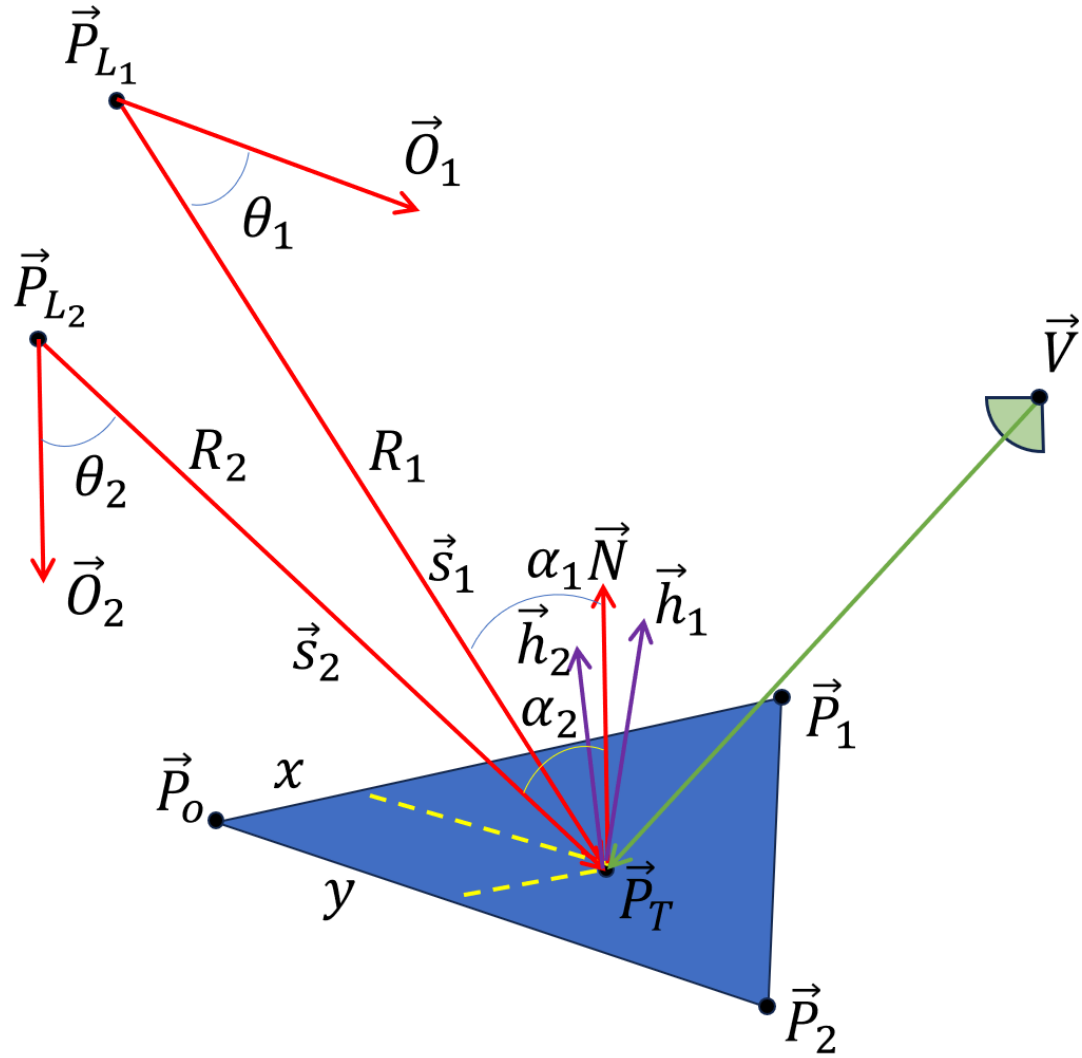
$L\left(RGB, \vec{P}_T, \vec{v}\right) = \frac{1}{\pi} \sum_{i \in lights} \left(E_i\left(RGB, \vec{P}_T\right) \bullet f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right)\right)$, *где* $E_i\left(RGB, \vec{P}_T\right)$ *- освещенность от i-го источника света,* $\vec{P}_T$ *- координаты точки,* $\vec{v}$ *- направление наблюдения,* $\vec{s}_i$ *- направление на источник света,* $f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right)$ *- двунаправленная функция отражения (BRDF),* $K(RGB)$ *– «цвет» поверхности.*

7. Двунаправленная функция отражения

$f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right) = K(RGB)\left(k_d + k_s\left(\vec{h} \bullet \vec{N}\right)^{k_e}\right)$, *где* k_d *- коэффициент диффузного отражения поверхности,* k_s *- коэффициент зеркального отражения поверхности,* k_e *– коэффициент, определяющий ширину блика,* $\vec{h}$ *- средний вектор между направлениями освещения и наблюдения,* $\vec{N}$ *- нормаль к поверхности в точке освещения.*

8. Средний вектор

$\vec{h} = \frac{\vec{v} + \vec{s}}{\|\vec{v} + \vec{s}\|}$.


$\vec{P}_{L_1}$
$\vec{O}_1$
θ_1
$\vec{P}_{L_2}$
$\vec{V}$
R_2
R_1
θ_2
$\vec{O}_2$
$\vec{s}_1$
α_1
$\vec{N}$
$\vec{h}_1$
$\vec{h}_2$
$\vec{s}_2$
α_2
$\vec{P}_1$
$\vec{P}_o$
x
y
$\vec{P}_T$
$\vec{P}_2$

- **Шаги алгоритма**

1. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{\left(\vec{P}_2 - \vec{P}_0\right) \times \left(\vec{P}_1 - \vec{P}_0\right)}{\left\|\left(\vec{P}_1 - \vec{P}_0\right) \times \left(\vec{P}_2 - \vec{P}_0\right)\right\|}$$

2. Перевод локальных координат точки в плоскости в глобальные

$\vec{P}_T = \vec{P}_0 + \left(\frac{\vec{P}_1 - \vec{P}_0}{\|\vec{P}_1 - \vec{P}_0\|} \cdot x + \frac{\vec{P}_2 - \vec{P}_0}{\|\vec{P}_2 - \vec{P}_0\|} \cdot y\right)$, *где* x *и* y *смещения по ребрам треугольника.*

3. Вектор от точки плоскости до источника света:

$$\vec{s} = \vec{P}_T - \vec{P}_L,\ R^2 = \|\vec{s}\|^2,\ \cos\alpha = \frac{\vec{s} \bullet \vec{N}}{\|\vec{s}\|},\ \cos\theta = \frac{\vec{s} \bullet \vec{O}}{\|\vec{s}\|}$$

4. «Цветная» сила излучения источника под углом к оси источника света:

$I\left(RGB, \vec{s}\right) = I_0(RGB)\cos\theta$, *где* $I_0(RGB)$ *– «цветная» сила излучения источника света в направлении его оси* $\vec{O}, \|\vec{O}\| = 1$, θ *- угол между направлением распространения света и осью источника света,* $\cos\theta$ *– диаграмма излучения.*

5. «Цветная» освещенность точки:

$E\left(RGB, \vec{P}_T\right) = \frac{I\left(RGB, \vec{s}\right)\cos\alpha}{R^2}$, *где* $I\left(RGB, \vec{s}\right)$ *– «цветная» сила излучения света,* α *- угол между направлением света и нормалью к освещаемой поверхности,* R *- расстояние от источника света до рассматриваемой точки.*

6. Средний вектор

$$\vec{h} = \frac{\vec{v} + \vec{s}}{\|\vec{v} + \vec{s}\|}.$$

7. Двунаправленная функция отражени

$f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right) = K(RGB)\left(k_d + k_s\left(\vec{h} \bullet \vec{N}\right)^{k_e}\right)$, *где* k_d *- коэффициент диффузного отражения поверхности,* k_s *- коэффициент зеркального отражения поверхности,* k_e *– коэффициент, определяющий ширину блика,* $\vec{h}$ *- средний вектор между направлениями освещения и наблюдения,* $\vec{N}$ *- нормаль к поверхности в точке освещения.*

8. Яркость точки:

$L\left(RGB, \vec{P}_T, \vec{v}\right) = \frac{1}{\pi} \sum_{i \in lights} \left(E_i\left(RGB, \vec{P}_T\right) \bullet f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right)\right)$, *где* $E_i\left(RGB, \vec{P}_T\right)$ *- освещенность от i-го источника света,* $\vec{P}_T$ *- координаты точки,* $\vec{v}$ *- направление наблюдения,* $\vec{s}_i$ *- направление на источник света,* $f\left(RGB, \vec{P}_T, \vec{v}, \vec{s}_i\right)$ *- двунаправленная функция отражения (BRDF),* $K(RGB)$ *– «цвет» поверхности.*

- **Программа для расчета яркости** (отдельный файл или файлы)

```
import tkinter as tk
from tkinter import messagebox, ttk
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import tkinter as tk
from tkinter import messagebox, ttk
import matplotlib.pyplot as plt
```

```python
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np

#  МАТЕМАТИЧЕСКИЕ ФУНКЦИИ

def clamp0(x):
   return max(0.0, x)

def dot3(a, b):
   return a[0] * b[0] + a[1] * b[1] + a[2] * b[2]


def norm3(v):
   return np.sqrt(dot3(v, v))


def normalize3(v):
   v = np.array(v, dtype=float)
   n = np.linalg.norm(v)
   return v if n == 0 else v / n


def cos_between(a, b):
   na = norm3(a)
   nb = norm3(b)
   if na == 0 or nb == 0:
       return 0.0
   return dot3(a, b) / (na * nb)


def rgb_mul(a, b):
   return np.array([a[0] * b[0], a[1] * b[1], a[2] * b[2]])


def rgb_add(a, b):
   return np.array([a[0] + b[0], a[1] + b[1], a[2] + b[2]])


class Light:
   def __init__(self, I0_rgb, O, PL, name="Light"):
       self.I0 = np.array(I0_rgb, dtype=float)
       self.O = normalize3(O)
       self.PL = np.array(PL, dtype=float)
       self.name = name


class Material:
   def __init__(self, K_rgb, kd, ks, ke):
       self.K = np.array(K_rgb, dtype=float)
       self.kd = kd
       self.ks = ks
       self.ke = ke


class Triangle:
   def __init__(self, P0, P1, P2):
       self.P0 = np.array(P0, dtype=float)
       self.P1 = np.array(P1, dtype=float)
```

```
        self.P2 = np.array(P2, dtype=float)
        self.N = self.normal_from_points(P0, P1, P2)

        # Для локальных координат используем барицентрические координаты
        self.v0 = self.P1 - self.P0  # вектор от P0 к P1
        self.v1 = self.P2 - self.P0  # вектор от P0 к P2

        # Для обратной совместимости (если где-то используется e1, e2)
        self.e1 = normalize3(self.v0)
        self.e2 = normalize3(self.v1)

    def normal_from_points(self, P0, P1, P2):
        """Вычисление нормали к треугольнику"""
        A = np.array(P2) - np.array(P0)
        B = np.array(P1) - np.array(P0)
        C = np.cross(A, B)
        return normalize3(C)

    def local_to_global(self, x, y):
        """Преобразование барицентрических координат (x,y) в глобальные
        x, y в диапазоне [0,1], где:
        (0,0) -> P0
        (1,0) -> P1
        (0,1) -> P2
        """
        # Барицентрические координаты: (x, y, 1-x-y)
        return self.P0 + self.v0 * x + self.v1 * y


def E_from_light(PT, I0, O, PL, N):
    L_vec = PT - PL
    dist = norm3(L_vec)
    if dist == 0:
        return np.zeros(3)

    L = normalize3(L_vec)
    cos_theta = clamp0(cos_between(N, -L))
    cos_alpha = clamp0(cos_between(O, L))
    falloff = 1.0 / (dist ** 2)

    return I0 * cos_theta * cos_alpha * falloff


def BRDF_f_rgb(K, N, V, S, kd, ks, ke):
    Nn = normalize3(N)
    Vn = normalize3(V)
    Sn = normalize3(S)

    if cos_between(Nn, Vn) <= 0.0 or cos_between(Nn, Sn) <= 0.0:
        return np.zeros(3)

    H = normalize3(Vn + Sn)
    h_dot_n = clamp0(cos_between(H, Nn))

    f_scalar = kd + ks * (h_dot_n ** ke)
    return K * f_scalar


def compute_brightness(lights, material, tri, PT, v_dir):
```

```
    N = tri.N
    v_dir = normalize3(v_dir)

    L = np.zeros(3)
    E_list = []

    for light in lights:
        E = E_from_light(PT, light.I0, light.O, light.PL, N)
        E_list.append(E)

        if np.any(E != 0):
            S = light.PL - PT
            f_rgb = BRDF_f_rgb(material.K, N, v_dir, S,
                               material.kd, material.ks, material.ke)
            L = L + rgb_mul(E, f_rgb) / np.pi

    return L, E_list


#  ВИЗУАЛИЗАЦИЯ

def render_view_from_direction(tri, lights, material, v_dir, resolution=300):
    """
    Изображение треугольника с точки зрения наблюдателя в глобальных
координатах
    """
    print("\n=== ВИЗУАЛИЗАЦИЯ С ТОЧКИ ЗРЕНИЯ НАБЛЮДАТЕЛЯ ===")
    print(f"Направление наблюдения V = [{v_dir[0]:.3f}, {v_dir[1]:.3f},
{v_dir[2]:.3f}]")
    print(f"Вершины треугольника:")
    print(f"  P0 = {tri.P0}")
    print(f"  P1 = {tri.P1}")
    print(f"  P2 = {tri.P2}")

    # Находим bounding box треугольника в глобальных координатах
    all_points = np.array([tri.P0, tri.P1, tri.P2])
    min_x, min_y = np.min(all_points[:, 0]), np.min(all_points[:, 1])
    max_x, max_y = np.max(all_points[:, 0]), np.max(all_points[:, 1])

    # Добавляем отступы
    padding_x = (max_x - min_x) * 0.1
    padding_y = (max_y - min_y) * 0.1

    # Создаем сетку в глобальных координатах X,Y
    x_vals = np.linspace(min_x - padding_x, max_x + padding_x, resolution)
    y_vals = np.linspace(min_y - padding_y, max_y + padding_y, resolution)

    brightness = np.zeros((resolution, resolution, 3))

    # Функция для проверки, находится ли точка внутри треугольника
    def point_in_triangle(px, py, p0, p1, p2):
        def sign(p1, p2, p3):
            return (p1[0] - p3[0]) * (p2[1] - p3[1]) - (p2[0] - p3[0]) * (p1[1] -
p3[1])

        d1 = sign((px, py), p1, p0)
        d2 = sign((px, py), p2, p1)
        d3 = sign((px, py), p0, p2)
```

```
    has_neg = (d1 < 0) or (d2 < 0) or (d3 < 0)
    has_pos = (d1 > 0) or (d2 > 0) or (d3 > 0)

    return not (has_neg and has_pos)

# Для каждой точки в глобальных координатах
for i, y in enumerate(y_vals):
    for j, x in enumerate(x_vals):
        # Проверяем, находится ли точка внутри треугольника
        if point_in_triangle(x, y, tri.P0[:2], tri.P1[:2], tri.P2[:2]):
            # Интерполируем Z координату (для освещения)
            # Находим барицентрические координаты для интерполяции Z
            # Упрощенный подход: используем площадь
            def barycentric(px, py, p0, p1, p2):
                v0 = np.array([p1[0] - p0[0], p1[1] - p0[1]])
                v1 = np.array([p2[0] - p0[0], p2[1] - p0[1]])
                v2 = np.array([px - p0[0], py - p0[1]])
                d00 = np.dot(v0, v0)
                d01 = np.dot(v0, v1)
                d11 = np.dot(v1, v1)
                d20 = np.dot(v2, v0)
                d21 = np.dot(v2, v1)
                denom = d00 * d11 - d01 * d01
                if abs(denom) < 1e-10:
                    return 0, 0, 1
                v = (d11 * d20 - d01 * d21) / denom
                w = (d00 * d21 - d01 * d20) / denom
                u = 1 - v - w
                return u, v, w

            u, v, w = barycentric(x, y, tri.P0[:2], tri.P1[:2], tri.P2[:2])
            z = u * tri.P0[2] + v * tri.P1[2] + w * tri.P2[2]

            PT = np.array([x, y, z])
            L_rgb, _ = compute_brightness(lights, material, tri, PT, v_dir)
            brightness[i, j] = L_rgb

# Нормализуем яркость в диапазон [0, 1]
brightness = np.clip(brightness, 0, 1)

# Создаем график
fig, ax = plt.subplots(1, 1, figsize=(12, 10))

# Отображаем RGB яркость
im = ax.imshow(brightness, origin='lower',
               extent=[min_x - padding_x, max_x + padding_x,
                       min_y - padding_y, max_y + padding_y],
               vmin=0, vmax=1, aspect='auto')

# Отмечаем вершины треугольника
ax.scatter([tri.P0[0], tri.P1[0], tri.P2[0]],
           [tri.P0[1], tri.P1[1], tri.P2[1]],
           color='red', s=150, zorder=5, edgecolors='black', linewidth=2,
           label='Вершины треугольника')

# Подписываем вершины с реальными координатами
ax.annotate(f'P0\n({tri.P0[0]:.2f}, {tri.P0[1]:.2f}, {tri.P0[2]:.2f})',
            (tri.P0[0], tri.P0[1]), xytext=(10, 10),
            textcoords='offset points', fontsize=9, fontweight='bold',
```

```
                bbox=dict(boxstyle='round,pad=0.3', facecolor='lightyellow',
alpha=0.8))

    ax.annotate(f'P1\n({tri.P1[0]:.2f}, {tri.P1[1]:.2f}, {tri.P1[2]:.2f})',
                (tri.P1[0], tri.P1[1]), xytext=(10, 10),
                textcoords='offset points', fontsize=9, fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='lightyellow',
alpha=0.8))

    ax.annotate(f'P2\n({tri.P2[0]:.2f}, {tri.P2[1]:.2f}, {tri.P2[2]:.2f})',
                (tri.P2[0], tri.P2[1]), xytext=(10, -15),
                textcoords='offset points', fontsize=9, fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.3', facecolor='lightyellow',
alpha=0.8))

    # Рисуем границы треугольника
    triangle_vertices = np.array([[tri.P0[0], tri.P0[1]],
                                  [tri.P1[0], tri.P1[1]],
                                  [tri.P2[0], tri.P2[1]],
                                  [tri.P0[0], tri.P0[1]]])
    ax.plot(triangle_vertices[:, 0], triangle_vertices[:, 1],
            'r--', alpha=0.5, linewidth=2, label='Границы треугольника')

    ax.set_title(
        f'Треугольник с точки зрения наблюдателя (Глобальные координаты X,Y)\n'
        f'Направление V = [{v_dir[0]:.3f}, {v_dir[1]:.3f}, {v_dir[2]:.3f}]\n'
        f'P0{tri.P0} | P1{tri.P1} | P2{tri.P2}',
        fontsize=11, fontweight='bold')
    ax.set_xlabel("X (глобальные координаты)", fontsize=12)
    ax.set_ylabel("Y (глобальные координаты)", fontsize=12)

    # Добавляем цветовую шкалу
    plt.colorbar(im, ax=ax, label='Яркость (нормализованная)')

    # Добавляем сетку
    ax.grid(True, alpha=0.3, linestyle='--')

    ax.legend(loc='upper right')
    ax.set_aspect('equal', adjustable='box')
    plt.tight_layout()
    plt.show()

def render_3d_scene(tri, lights, v_dir, material):
    """
    3D сцена с местоположением источников, наблюдателя и треугольника
    """
    print("\n=== 3D СЦЕНА ===")

    fig = plt.figure(figsize=(14, 10))
    ax = fig.add_subplot(111, projection='3d')

    # Рисуем треугольник
    vertices = np.array([tri.P0, tri.P1, tri.P2])
    triangle = Poly3DCollection([vertices], alpha=0.6, edgecolor='black',
linewidth=2)

    # Цвет треугольника на основе материала
    triangle_color = material.K / np.max(material.K) if np.max(material.K) > 0
else material.K
```

```
    triangle.set_facecolor(triangle_color)
    ax.add_collection3d(triangle)

    # Добавляем вершины треугольника
    ax.scatter(vertices[:, 0], vertices[:, 1], vertices[:, 2],
               color='red', s=100, zorder=5)

    # Подписываем вершины
    ax.text(tri.P0[0], tri.P0[1], tri.P0[2], ' P0', fontsize=10, color='red')
    ax.text(tri.P1[0], tri.P1[1], tri.P1[2], ' P1', fontsize=10, color='red')
    ax.text(tri.P2[0], tri.P2[1], tri.P2[2], ' P2', fontsize=10, color='red')

    # Рисуем нормаль треугольника
    center = (tri.P0 + tri.P1 + tri.P2) / 3
    normal_length = 0.5
    ax.quiver(center[0], center[1], center[2],
              tri.N[0] * normal_length, tri.N[1] * normal_length, tri.N[2] *
normal_length,
              color='purple', linewidth=2, arrow_length_ratio=0.3,
label='Нормаль')

    # Рисуем источники света
    light_colors = ['yellow', 'orange']
    for i, light in enumerate(lights):
        # Источник света
        ax.scatter(light.PL[0], light.PL[1], light.PL[2],
                   color=light_colors[i], s=200, zorder=5,
                   edgecolor='black', linewidth=2)

        # Подпись источника
        ax.text(light.PL[0], light.PL[1], light.PL[2], f' {light.name}',
                fontsize=11, color=light_colors[i], fontweight='bold')

        # Направление оси источника
        axis_end = light.PL + light.O * 1.5
        ax.quiver(light.PL[0], light.PL[1], light.PL[2],
                  light.O[0] * 1.5, light.O[1] * 1.5, light.O[2] * 1.5,
                  color=light_colors[i], linewidth=2, arrow_length_ratio=0.25,
                  alpha=0.7)

        # Линии от источника к центру треугольника
        ax.plot([light.PL[0], center[0]],
                [light.PL[1], center[1]],
                [light.PL[2], center[2]],
                '--', color=light_colors[i], alpha=0.4, linewidth=1)

    # Рисуем наблюдателя
    # Наблюдатель находится в направлении V от центра треугольника
    observer_pos = center + v_dir * 3
    ax.scatter(observer_pos[0], observer_pos[1], observer_pos[2],
               color='cyan', s=150, zorder=5, edgecolor='black', linewidth=2)
    ax.text(observer_pos[0], observer_pos[1], observer_pos[2], ' Наблюдатель',
            fontsize=11, color='cyan', fontweight='bold')

    # Направление взгляда наблюдателя (к центру треугольника)
    view_dir = center - observer_pos
    ax.quiver(observer_pos[0], observer_pos[1], observer_pos[2],
              view_dir[0], view_dir[1], view_dir[2],
              color='cyan', linewidth=2, arrow_length_ratio=0.3, alpha=0.8)
```

```
    # Оси координат
    # Ось X - красная
    ax.quiver(0, 0, 0, 3, 0, 0, color='red', linewidth=2, arrow_length_ratio=0.2)
    ax.text(3, 0, 0, ' X', fontsize=12, color='red', fontweight='bold')

    # Ось Y - зеленая
    ax.quiver(0, 0, 0, 0, 3, 0, color='green', linewidth=2,
arrow_length_ratio=0.2)
    ax.text(0, 3, 0, ' Y', fontsize=12, color='green', fontweight='bold')

    # Ось Z - синяя
    ax.quiver(0, 0, 0, 0, 0, 3, color='blue', linewidth=2,
arrow_length_ratio=0.2)
    ax.text(0, 0, 3, ' Z', fontsize=12, color='blue', fontweight='bold')

    # Настройка отображения
    ax.set_xlabel('X', fontsize=12)
    ax.set_ylabel('Y', fontsize=12)
    ax.set_zlabel('Z', fontsize=12)
    ax.set_title('3D Сцена: треугольник, источники света и наблюдатель',
fontsize=14, fontweight='bold')

    # Легенда
    legend_elements = [
        plt.Line2D([0], [0], color='red', linewidth=2, label='Ось X'),
        plt.Line2D([0], [0], color='green', linewidth=2, label='Ось Y'),
        plt.Line2D([0], [0], color='blue', linewidth=2, label='Ось Z'),
        plt.Line2D([0], [0], color='purple', linewidth=2, label='Нормаль
треугольника'),
        plt.Line2D([0], [0], color='yellow', linewidth=2, label='Источник 1'),
        plt.Line2D([0], [0], color='orange', linewidth=2, label='Источник 2'),
        plt.Line2D([0], [0], color='cyan', linewidth=2, label='Наблюдатель'),
        plt.Rectangle((0, 0), 1, 1, facecolor=triangle_color, alpha=0.6,
label='Треугольник')
    ]
    ax.legend(handles=legend_elements, loc='upper right')

    # Устанавливаем равные масштабы
    max_range = max(np.max(vertices[:, 0]) - np.min(vertices[:, 0]),
                    np.max(vertices[:, 1]) - np.min(vertices[:, 1]),
                    np.max(vertices[:, 2]) - np.min(vertices[:, 2])) * 1.5

    mid_x = (np.max(vertices[:, 0]) + np.min(vertices[:, 0])) / 2
    mid_y = (np.max(vertices[:, 1]) + np.min(vertices[:, 1])) / 2
    mid_z = (np.max(vertices[:, 2]) + np.min(vertices[:, 2])) / 2

    ax.set_xlim(mid_x - max_range, mid_x + max_range)
    ax.set_ylim(mid_y - max_range, mid_y + max_range)
    ax.set_zlim(mid_z - max_range, mid_z + max_range)

    plt.tight_layout()
    plt.show()


#  GUI

class SimpleApp:
    def __init__(self, root):
```

```
        self.root = root
        root.title("Расчет освещенности и яркости")
        root.geometry("1000x750")

        # Создаем вкладки
        notebook = ttk.Notebook(root)
        notebook.pack(fill='both', expand=True, padx=5, pady=5)

        # Вкладка с параметрами
        self.params_frame = ttk.Frame(notebook)
        notebook.add(self.params_frame, text="Параметры")

        # Вкладка с результатами
        self.results_frame = ttk.Frame(notebook)
        notebook.add(self.results_frame, text="Результаты")

        # Создаем поля ввода
        self.create_input_fields()

        # Текстовое поле для результатов
        self.text_result = tk.Text(self.results_frame, wrap='none',
font=('Courier', 9))
        self.text_result.pack(fill='both', expand=True)

        # Скроллбары
        scroll_y = ttk.Scrollbar(self.results_frame, orient='vertical',
command=self.text_result.yview)
        scroll_y.pack(side='right', fill='y')
        scroll_x = ttk.Scrollbar(self.results_frame, orient='horizontal',
command=self.text_result.xview)
        scroll_x.pack(side='bottom', fill='x')
        self.text_result.configure(yscrollcommand=scroll_y.set,
xscrollcommand=scroll_x.set)

        # Кнопки
        btn_frame = ttk.Frame(root)
        btn_frame.pack(pady=10)

        ttk.Button(btn_frame, text="Выполнить расчет",
command=self.calculate).pack(side='left', padx=5)
        ttk.Button(btn_frame, text="Очистить",
command=self.clear_results).pack(side='left', padx=5)
        ttk.Button(btn_frame, text="Вид наблюдателя",
command=self.visualize_view).pack(side='left', padx=5)
        ttk.Button(btn_frame, text="3D сцена",
command=self.visualize_3d).pack(side='left', padx=5)

        # Статус
        self.status = tk.StringVar()
        self.status.set("Готов")
        status_label = ttk.Label(root, textvariable=self.status, relief='sunken')
        status_label.pack(side='bottom', fill='x')

    def create_input_fields(self):
        # Создаем canvas с прокруткой для параметров
        canvas = tk.Canvas(self.params_frame)
        scrollbar = ttk.Scrollbar(self.params_frame, orient='vertical',
command=canvas.yview)
        scrollable_frame = ttk.Frame(canvas)
```

```
        scrollable_frame.bind(
            "<Configure>",
            lambda e: canvas.configure(scrollregion=canvas.bbox("all"))
        )

        canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")
        canvas.configure(yscrollcommand=scrollbar.set)

        canvas.pack(side='left', fill='both', expand=True)
        scrollbar.pack(side='right', fill='y')

        # Поля ввода
        row = 0
        self.entries = {}

        # Источник 1
        ttk.Label(scrollable_frame, text="ИСТОЧНИК 1", font=('Arial', 10,
'bold')).grid(row=row, column=0, columnspan=6,

pady=(10, 5), sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "I01 (RGB)", ["R",
"G", "B"], [100, 100, 100], "I01")
        row = self.add_vector_entry(scrollable_frame, row, "O1 (направление)",
["x", "y", "z"], [0, 0, -1], "O1")
        row = self.add_vector_entry(scrollable_frame, row, "PL1 (позиция)", ["x",
"y", "z"], [0, 0, 8], "PL1")

        # Источник 2
        ttk.Label(scrollable_frame, text="ИСТОЧНИК 2", font=('Arial', 10,
'bold')).grid(row=row, column=0, columnspan=6,

pady=(10, 5), sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "I02 (RGB)", ["R",
"G", "B"], [80, 80, 80], "I02")
        row = self.add_vector_entry(scrollable_frame, row, "O2 (направление)",
["x", "y", "z"], [-0.5, 0, -1], "O2")
        row = self.add_vector_entry(scrollable_frame, row, "PL2 (позиция)", ["x",
"y", "z"], [4, 2, 7], "PL2")

        # Треугольник
        ttk.Label(scrollable_frame, text="ТРЕУГОЛЬНИК", font=('Arial', 10,
'bold')).grid(row=row, column=0,

columnspan=6, pady=(10, 5),

sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "P0", ["x", "y", "z"],
[0, 0, 0], "P0")
        row = self.add_vector_entry(scrollable_frame, row, "P1", ["x", "y", "z"],
[0, 2, 0], "P1")
        row = self.add_vector_entry(scrollable_frame, row, "P2", ["x", "y", "z"],
[2, 0, 0], "P2")
```

```
        # Точки (5 точек)
        ttk.Label(scrollable_frame, text="ТОЧКИ НА ТРЕУГОЛЬНИКЕ (локальные
координаты x,y)",
                  font=('Arial', 10, 'bold')).grid(row=row, column=0,
columnspan=6, pady=(10, 5), sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "Точка 1", ["x", "y"],
[0.2, 0.2], "p1", is_2d=True)
        row = self.add_vector_entry(scrollable_frame, row, "Точка 2", ["x", "y"],
[0.6, 0.2], "p2", is_2d=True)
        row = self.add_vector_entry(scrollable_frame, row, "Точка 3", ["x", "y"],
[0.2, 0.6], "p3", is_2d=True)
        row = self.add_vector_entry(scrollable_frame, row, "Точка 4", ["x", "y"],
[0.9, 0.4], "p4", is_2d=True)
        row = self.add_vector_entry(scrollable_frame, row, "Точка 5", ["x", "y"],
[0.4, 0.9], "p5", is_2d=True)

        # Материал
        ttk.Label(scrollable_frame, text="МАТЕРИАЛ", font=('Arial', 10,
'bold')).grid(row=row, column=0, columnspan=6,

pady=(10, 5), sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "K (цвет RGB)", ["R",
"G", "B"], [1.0, 0.6, 0.6], "K")
        row = self.add_scalar_entry(scrollable_frame, row, "kd (диффузный)", 0.6,
"kd")
        row = self.add_scalar_entry(scrollable_frame, row, "ks (зеркальный)",
0.4, "ks")
        row = self.add_scalar_entry(scrollable_frame, row, "ke (глянец)", 20.0,
"ke")

        # Наблюдатель
        ttk.Label(scrollable_frame, text="НАБЛЮДАТЕЛЬ", font=('Arial', 10,
'bold')).grid(row=row, column=0,

columnspan=6, pady=(10, 5),

sticky='w')
        row += 1

        row = self.add_vector_entry(scrollable_frame, row, "V (направление)",
["x", "y", "z"], [0, 0, 1], "V")

        # Пустое место внизу
        ttk.Label(scrollable_frame, text="").grid(row=row, column=0, pady=20)

    def add_vector_entry(self, parent, row, label, components, default, key,
is_2d=False):
        ttk.Label(parent, text=label, width=18, anchor='w').grid(row=row,
column=0, padx=5, pady=2, sticky='w')

        entries = []
        for i, comp in enumerate(components):
            ttk.Label(parent, text=comp, width=2).grid(row=row, column=1 + i * 2,
padx=2)
```

```
            var = tk.StringVar(value=str(default[i]))
            entry = ttk.Entry(parent, textvariable=var, width=8)
            entry.grid(row=row, column=2 + i * 2, padx=2, pady=2)
            entries.append(var)

        self.entries[key] = entries
        return row + 1

    def add_scalar_entry(self, parent, row, label, default, key):
        ttk.Label(parent, text=label, width=18, anchor='w').grid(row=row,
column=0, padx=5, pady=2, sticky='w')
        var = tk.StringVar(value=str(default))
        entry = ttk.Entry(parent, textvariable=var, width=8)
        entry.grid(row=row, column=1, padx=5, pady=2, sticky='w')
        self.entries[key] = var
        return row + 1

    def get_float(self, val):
        return float(val.replace(',', '.'))

    def calculate(self):
        try:
            # Считываем данные
            I01 = np.array([self.get_float(v.get()) for v in
self.entries["I01"]])
            I02 = np.array([self.get_float(v.get()) for v in
self.entries["I02"]])
            O1 = np.array([self.get_float(v.get()) for v in self.entries["O1"]])
            O2 = np.array([self.get_float(v.get()) for v in self.entries["O2"]])
            PL1 = np.array([self.get_float(v.get()) for v in
self.entries["PL1"]])
            PL2 = np.array([self.get_float(v.get()) for v in
self.entries["PL2"]])
            P0 = np.array([self.get_float(v.get()) for v in self.entries["P0"]])
            P1 = np.array([self.get_float(v.get()) for v in self.entries["P1"]])
            P2 = np.array([self.get_float(v.get()) for v in self.entries["P2"]])

            # Точки
            p1 = (self.get_float(self.entries["p1"][0].get()),
self.get_float(self.entries["p1"][1].get()))
            p2 = (self.get_float(self.entries["p2"][0].get()),
self.get_float(self.entries["p2"][1].get()))
            p3 = (self.get_float(self.entries["p3"][0].get()),
self.get_float(self.entries["p3"][1].get()))
            p4 = (self.get_float(self.entries["p4"][0].get()),
self.get_float(self.entries["p4"][1].get()))
            p5 = (self.get_float(self.entries["p5"][0].get()),
self.get_float(self.entries["p5"][1].get()))

            V = np.array([self.get_float(v.get()) for v in self.entries["V"]])
            K = np.array([self.get_float(v.get()) for v in self.entries["K"]])
            kd = self.get_float(self.entries["kd"].get())
            ks = self.get_float(self.entries["ks"].get())
            ke = self.get_float(self.entries["ke"].get())

            # Создаем объекты
            self.lights = [Light(I01, O1, PL1, "Источник 1"), Light(I02, O2, PL2,
"Источник 2")]
            self.tri = Triangle(P0, P1, P2)
```

```
            self.material = Material(K, kd, ks, ke)
            self.V = V

            # Точки для расчета
            x_vals = [p1[0], p2[0], p3[0], p4[0], p5[0]]
            y_vals = [p1[1], p2[1], p3[1], p4[1], p5[1]]

            # Расчет
            self.E1_matrix = []
            self.E2_matrix = []
            self.L_matrix = []

            for y in y_vals:
                row_e1, row_e2, row_l = [], [], []
                for x in x_vals:
                    PT = self.tri.local_to_global(x, y)
                    L_rgb, E_list = compute_brightness(self.lights,
self.material, self.tri, PT, V)
                    row_e1.append(E_list[0])
                    row_e2.append(E_list[1])
                    row_l.append(L_rgb)
                self.E1_matrix.append(row_e1)
                self.E2_matrix.append(row_e2)
                self.L_matrix.append(row_l)

            # Вывод результатов
            self.show_results(x_vals, y_vals)
            self.status.set("Расчет выполнен успешно")

        except Exception as e:
            messagebox.showerror("Ошибка", str(e))
            self.status.set("Ошибка расчета")

    def show_results(self, x_vals, y_vals):
        self.text_result.delete('1.0', tk.END)

        # Таблица E1
        self.text_result.insert(tk.END, "=" * 100 + "\n")
        self.text_result.insert(tk.END, "ТАБЛИЦА 1: Освещенность E1 от источника
1\n")
        self.text_result.insert(tk.END, "=" * 100 + "\n")

        # Заголовок
        header = "y\\x    "
        for x in x_vals:
            header += f"{x:^30}"
        self.text_result.insert(tk.END, header + "\n")
        self.text_result.insert(tk.END, "-" * 100 + "\n")

        for i, y in enumerate(y_vals):
            row = f"{y:.3f}  "
            for j in range(len(x_vals)):
                val = self.E1_matrix[i][j]
                row += f"({val[0]:.4f},{val[1]:.4f},{val[2]:.4f}) "
            self.text_result.insert(tk.END, row + "\n")

        # Таблица E2
        self.text_result.insert(tk.END, "\n" + "=" * 100 + "\n")
```

```
        self.text_result.insert(tk.END, "ТАБЛИЦА 2: Освещенность E2 от источника
2\n")
        self.text_result.insert(tk.END, "=" * 100 + "\n")

        self.text_result.insert(tk.END, header + "\n")
        self.text_result.insert(tk.END, "-" * 100 + "\n")

        for i, y in enumerate(y_vals):
            row = f"{y:.3f}  "
            for j in range(len(x_vals)):
                val = self.E2_matrix[i][j]
                row += f"({val[0]:.4f},{val[1]:.4f},{val[2]:.4f}) "
            self.text_result.insert(tk.END, row + "\n")

        # Таблица L
        self.text_result.insert(tk.END, "\n" + "=" * 100 + "\n")
        self.text_result.insert(tk.END, "ТАБЛИЦА 3: Яркость L\n")
        self.text_result.insert(tk.END, "=" * 100 + "\n")

        self.text_result.insert(tk.END, header + "\n")
        self.text_result.insert(tk.END, "-" * 100 + "\n")

        for i, y in enumerate(y_vals):
            row = f"{y:.3f}  "
            for j in range(len(x_vals)):
                val = self.L_matrix[i][j]
                row += f"({val[0]:.4f},{val[1]:.4f},{val[2]:.4f}) "
            self.text_result.insert(tk.END, row + "\n")

    def clear_results(self):
        self.text_result.delete('1.0', tk.END)
        self.status.set("Результаты очищены")

    def visualize_view(self):
        if hasattr(self, 'tri'):
            try:
                render_view_from_direction(self.tri, self.lights, self.material,
self.V)
            except Exception as e:
                messagebox.showerror("Ошибка визуализации", str(e))
        else:
            messagebox.showwarning("Предупреждение", "Сначала выполните расчет")

    def visualize_3d(self):
        if hasattr(self, 'tri'):
            try:
                render_3d_scene(self.tri, self.lights, self.V, self.material)
            except Exception as e:
                messagebox.showerror("Ошибка 3D визуализации", str(e))
        else:
            messagebox.showwarning("Предупреждение", "Сначала выполните расчет")


# Запуск приложения
if __name__ == "__main__":
    root = tk.Tk()
    app = SimpleApp(root)
    root.mainloop()
```

- **Пример входных данных и результаты**

1. $I_{0_1}(RGB) = (1.0,\ 0.8,\ 0.7) I_{0_2}(RGB) = (0.5,\ 0.7,\ 1.0)$

$\vec{O}_1 = (0.0,\ 0.0,\ 1.0) \vec{O}_2 = (0.0,\ 0.0,\ 1.0)$

$\vec{P}_{L_1} = (5.0,\ 5.0,\ -3.0) \vec{P}_{L_2} = (-4.0,\ 2.0,\ 4.0)$

2. $\vec{P}_0 = (0.0,\ 0.0,\ 0.0)$
3. $\vec{P}_1 = (0.0,\ 1.0,\ 0.0)$
4. $\vec{P}_2 = (1.0,\ 0.0,\ 0.0)$

$x_1 = 0.1,\ y_1 = 0.1$
$x_2 = 0.2,\ y_2 = 0.2$

$x_5 = 0.3,\ y_5 = 0.3$

$x_4 = 0.4,\ y_4 = 0.4$

$x_5 = 0.5,\ y_5 = 0.5$

5. $\vec{V} = (0.0,\ 0.0,\ 1.0)$
6. $K(RGB) = (0.8,\ 0.8,\ 0.8)$
7. $k_d = 0.7$
8. $k_s = 0.3$
9. $k_e = 20.0$

Вычисленные значения освещенности $E_1\left(RGB, \vec{P}_T\right)$ для точек, заданных локальными координатами:

y \ x	0.1	0.2	0.3	0.4	0.5
0.1	0.0028,0.0022,0.0019	0.0029,0.0023,0.0020	0.0030,0.0024,0.0021	0.0031,0.0025,0.0021	0.0032,0.0025,0.0022
0.2	0.0029,0.0023,0.0020	0.0030,0.0024,0.0021	0.0031,0.0025,0.0022	0.0032,0.0025,0.0022	0.0033,0.0026,0.0023
0.3	0.0030,0.0024,0.0021	0.0031,0.0025,0.0022	0.0032,0.0025,0.0022	0.0033,0.0026,0.0023	0.0034,0.0027,0.0024
0.4	0.0031,0.0025,0.0021	0.0032,0.0025,0.0022	0.0033,0.0026,0.0023	0.0034,0.0027,0.0024	0.0035,0.0028,0.0025
0.5	0.0032,0.0025,0.0022	0.0033,0.0026,0.0023	0.0034,0.0027,0.0024	0.0035,0.0028,0.0025	0.0037,0.0029,0.0026

Вычисленные значения освещенности $E_2\left(\left(RGB, \vec{P}_T\right)\right)$ для тех же точек, заданных локальными координатами:

y \ x	0.1	0.2	0.3	0.4	0.5
0.1	0.0060,0.0084,0.0121	0.0062,0.0086,0.0123	0.0063,0.0088,0.0126	0.0064,0.0090,0.0128	0.0065,0.0091,0.0130
0.2	0.0058,0.0081,0.0115	0.0059,0.0082,0.0118	0.0060,0.0084,0.0120	0.0061,0.0085,0.0122	0.0062,0.0087,0.0124
0.3	0.0055,0.0077,0.0110	0.0056,0.0079,0.0112	0.0057,0.0080,0.0115	0.0058,0.0082,0.0117	0.0059,0.0083,0.0119
0.4	0.0053,0.0074,0.0105	0.0054,0.0075,0.0107	0.0055,0.0077,0.0109	0.0056,0.0078,0.0111	0.0057,0.0079,0.0113
0.5	0.0050,0.0070,0.0101	0.0051,0.0072,0.0103	0.0052,0.0073,0.0104	0.0053,0.0074,0.0106	0.0054,0.0076,0.0108

Вычисленные значения яркостей $L\left(RGB, \vec{P}_T, \vec{v}\right)$ для тех же точек на плоскости с заданными условиями наблюдения

y \ x	0.1	0.2	0.3	0.4	0.5
0.1	0.0016,0.0020,0.0026	0.0017,0.0021,0.0027	0.0017,0.0021,0.0028	0.0018,0.0022,0.0028	0.0018,0.0022,0.0029
0.2	0.0016,0.0019,0.0026	0.0017,0.0020,0.0026	0.0017,0.0020,0.0027	0.0017,0.0021,0.0027	0.0018,0.0021,0.0028
0.3	0.0016,0.0019,0.0025	0.0016,0.0019,0.0025	0.0017,0.0020,0.0026	0.0017,0.0020,0.0026	0.0017,0.0021,0.0027
0.4	0.0015,0.0018,0.0024	0.0016,0.0019,0.0024	0.0016,0.0019,0.0025	0.0017,0.0020,0.0025	0.0017,0.0020,0.0026
0.5	0.0015,0.0018,0.0023	0.0016,0.0018,0.0024	0.0016,0.0019,0.0024	0.0016,0.0019,0.0025	0.0017,0.0020,0.0025

- **Вывод**

В данной работе был изучен и реализован алгоритм вычисления яркости точки на плоскости с учетом освещенности от нескольких источников света. Используя заданные входные данные, программа корректно вычисляет значения освещенности и яркости для различных точек плоскости, учитывая направленность света и свойства поверхности.

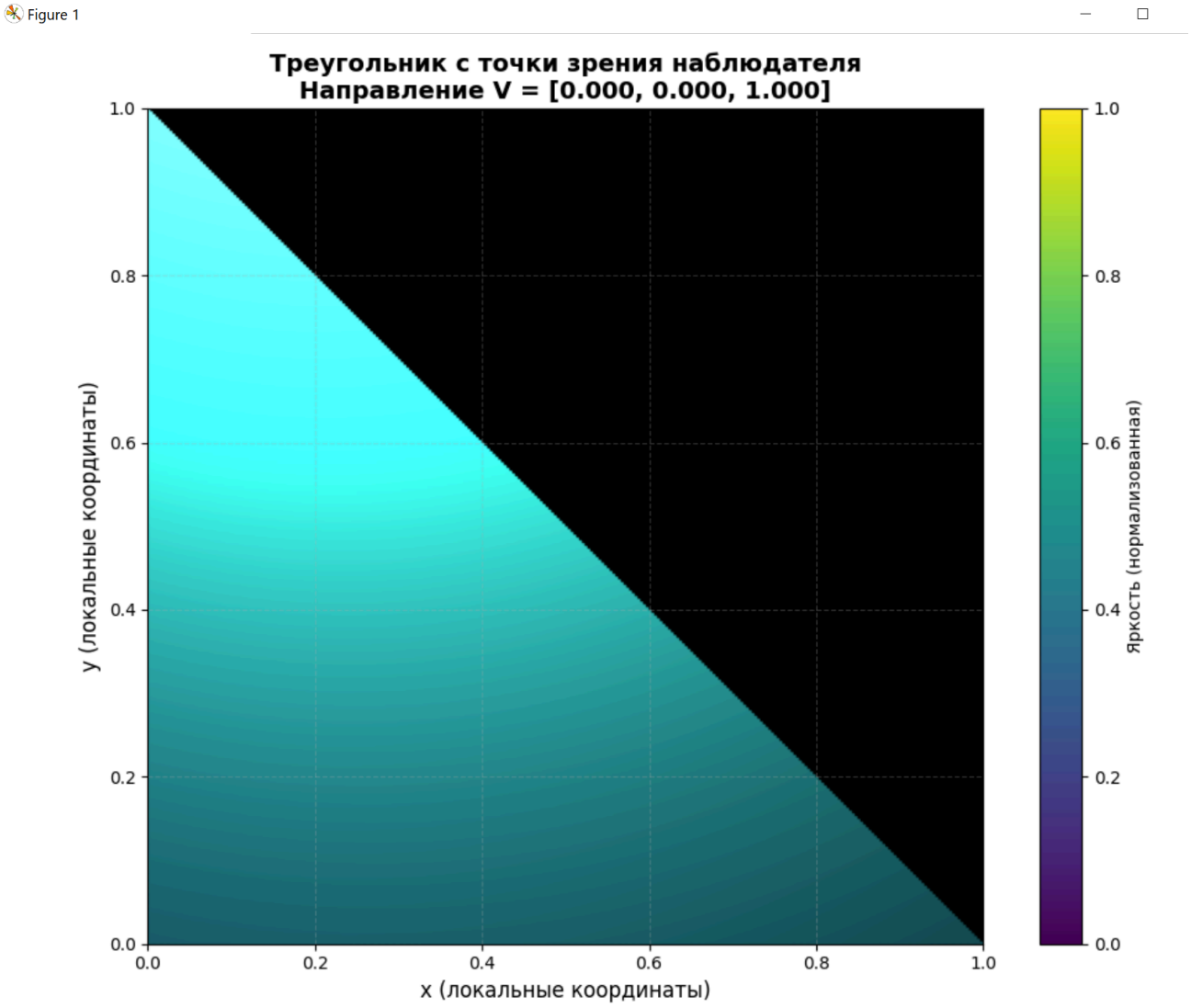
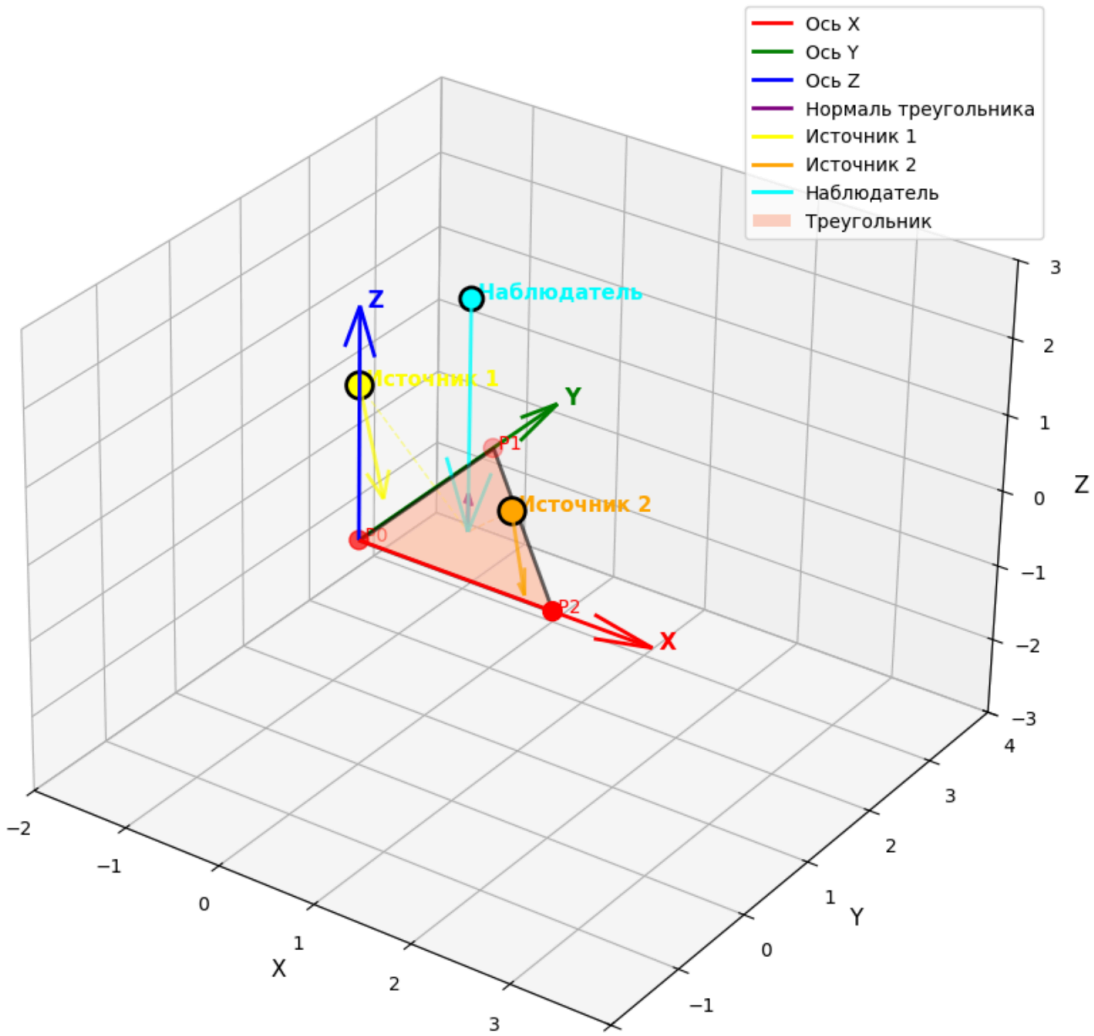


Figure 1
Треугольник с точки зрения наблюдателя
Направление V = [0.000, 0.000, 1.000]
y (локальные координаты)
x (локальные координаты)
Яркость (нормализованная)
0.0
0.2
0.4
0.6
0.8
1.0


Ось X
Ось Y
Ось Z
Нормаль треугольника
Источник 1
Источник 2
Наблюдатель
Треугольник
Z
Наблюдатель
Источник 1
Y
P1
Источник 2
P0
P2
X
X
Y
Z